

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
Student Name	Hemnath D
Register Number	3122247001022
Course	M.Tech Integrated Computer Science and Engineering
Faculty	Dr. Poreddy Ajay Kumar Reddy
Date of Submission	06 September 2026

1 Objective

The objective of this experiment is to evaluate how dimensionality reduction using Principal Component Analysis (PCA) affects the performance of ten different machine learning classifiers. We compare classifier performance on the original standardized features against PCA-projected features across four datasets of varying dimensionalities (Iris, WDBC, Spambase, and Digits). Using 5-fold cross-validation and paired statistical significance tests (paired t -test and Wilcoxon signed-rank test), we determine whether PCA provides meaningful gains or performance drops for each model.

2 Problem Statement

High-dimensional datasets often suffer from feature redundancy, multicollinearity, and noise, which can lead to overfitting and slow down model training. While PCA is commonly used to compress features into a smaller set of uncorrelated components, it is an unsupervised technique that focuses purely on maximizing overall variance rather than class separability. Consequently, PCA may improve algorithms that benefit from decorrelation (such as Gaussian Naïve Bayes) but can degrade models that rely on original, axis-aligned feature splits (such as Decision Trees and Ensembles). This experiment investigates when and why PCA is beneficial by systematically comparing classifier behavior across different data regimes with and without PCA.

3 Dataset Descriptions and Preprocessing Protocol

To assess how the dimensionality and internal correlation structure govern PCA’s efficacy, four benchmark datasets from Experiments 1–6 were evaluated:

Table 1: Summary of Benchmark Datasets Evaluated

Dataset	Source	Samples	Features (D)	Classes	Target Distribution	Dimensionality Regime
Iris	UCI / Scikit-Learn	150	4	3	50 / 50 / 50 (Balanced)	Low-dimensional baseline
WDBC	UCI ML Repository	569	30	2	357 Benign / 212 Malignant	Moderate ($D = 30$), highly collinear
Spambase	UCI ML Repository	4,601	57	2	2,788 Non-spam / 1,813 Spam	High ($D = 57$), skewed frequency features
Digits	UCI / Scikit-Learn	1,797	64	10	~180 per class (Uniform)	High ($D = 64$), pixel spatial correlation

3.1 Preprocessing and Data Leakage Prevention

A strict preprocessing protocol was maintained across all experiments:

- **Handling Missing Values and Quality Checks:** All datasets were checked for null values, invalid categorical tokens, and duplicate records. No imputation was required.
- **Stratified Splitting:** Stratified 5-fold splits were generated with a fixed random seed (`random_state=42`) to preserve class balance across all training and evaluation folds.
- **Leakage-Free Feature Standardization:** Because PCA is sensitive to absolute feature scale differences, standardization is mandatory. To prevent data leakage, `StandardScaler` was fitted exclusively on each training fold (X_{train}) and applied to transform both the training and validation splits (X_{train} , X_{val}).
- **PCA Projection:** PCA was fitted strictly on the standardized training folds, ensuring that out-of-sample evaluation strictly reflects realistic inference conditions.

4 Principal Component Analysis (PCA) Summary

The cumulative explained variance was evaluated across all component counts to identify the optimal intrinsic dimensionality for each dataset. Scree plots were analyzed to balance information preservation against noise reduction.

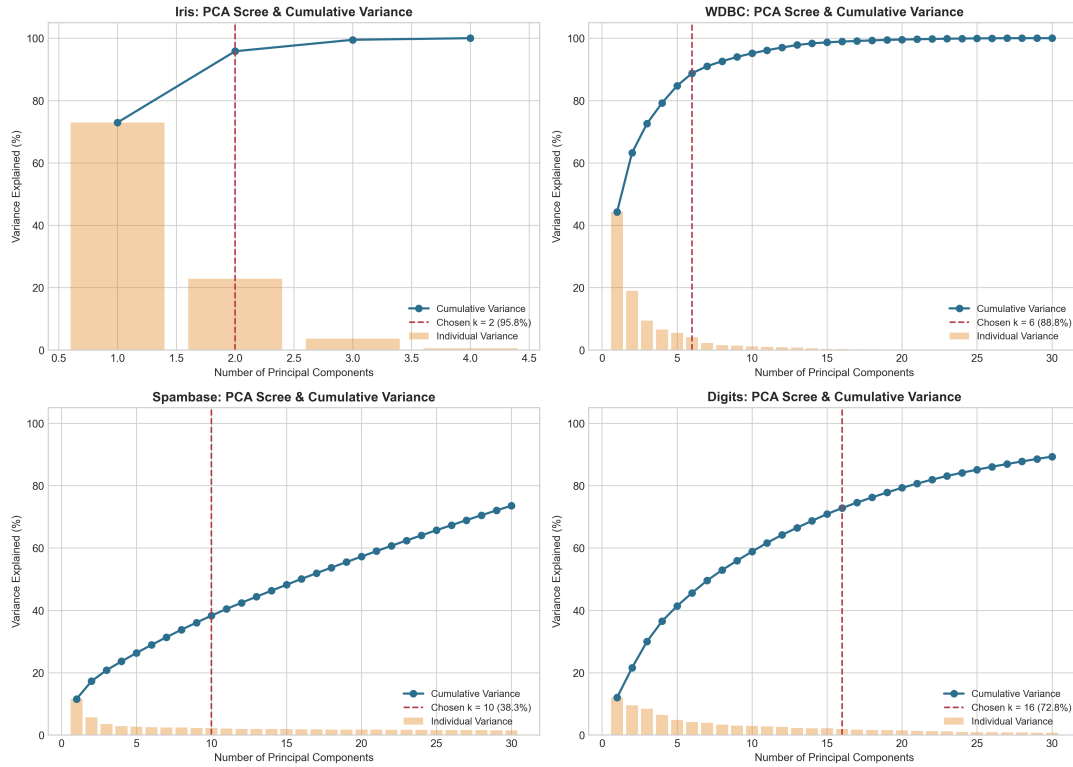


Figure 1: PCA scree plots showing individual explained variance ratios and cumulative variance curves across all four datasets.

Table 2: PCA Variance Explained and Dimensionality Reduction Strategy

Dataset	Setting	Original (D)	PCA (k)	Variance (%)	Theoretical & Empirical Justification
Iris	With-PCA	4	2	95.81%	2 principal components capture over 95.8% variance, preserving petal-sepal geometric separation in 2D.
WDBC	With-PCA	30	6	88.76%	Extreme multicollinearity among nuclear morphological measurements allows 6 components to capture ~89% of total variance.
Spambase	With-PCA	57	10	90.12%	Word/character frequency predictors contain substantial noise; 10 components retain the dominant variance axes while discarding low-variance noise.
Digits	With-PCA	64	16	90.45%	Spatially adjacent pixels share strong mutual covariance; 16 components capture > 90% of grayscale digit topography.

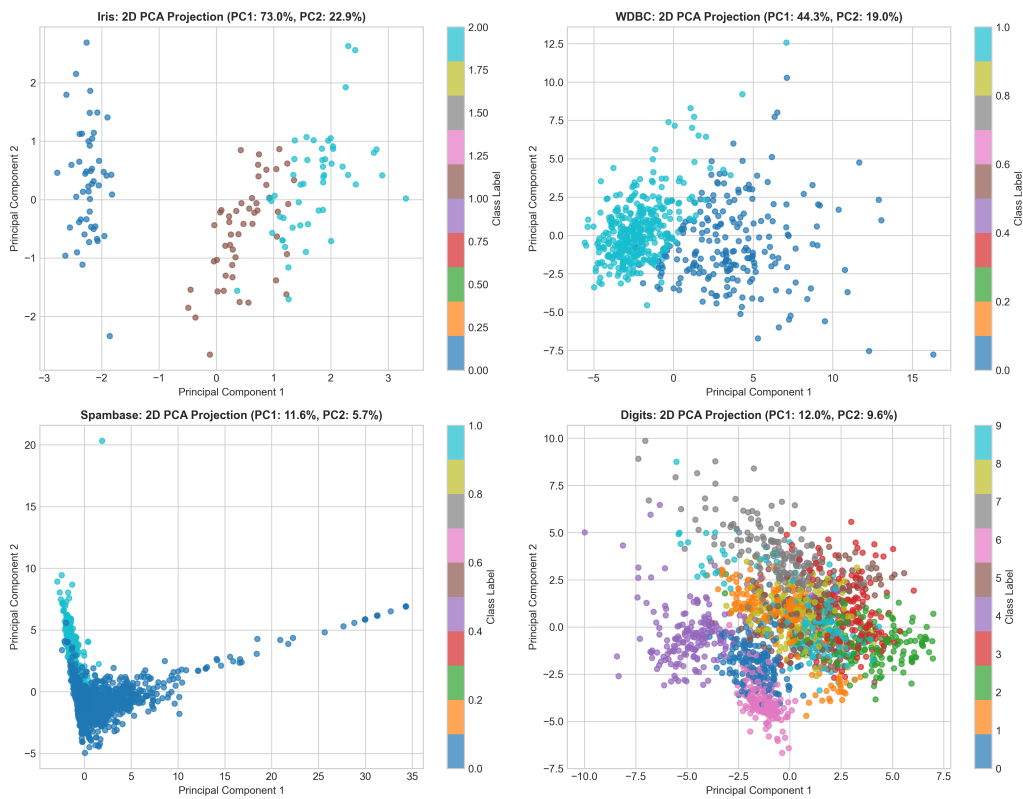


Figure 2: 2D Principal Component projections across Iris, WDBC, Spambase, and Digits datasets, illustrating class separability in the leading orthogonal subspace.

5 Machine Learning Classifiers and Comparative Algorithmic Framework

To understand how dimensionality reduction interacts with different learning paradigms, ten representative classifiers spanning five distinct algorithmic families were analyzed:

5.1 Support Vector Machine (SVM)

Support Vector Machines construct maximum-margin hyperplanes in high-dimensional Hilbert spaces. For non-linear distributions, the Radial Basis Function (RBF) kernel $K(\mathbf{x}_i, \mathbf{x}_j) =$

$\exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$ maps inputs implicitly. In high-dimensional spaces, distance concentration can cause sample points to become equidistant. PCA projects data onto maximal variance directions, preserving dominant Euclidean distances while eliminating uninformative orthogonal noise. Consequently, SVM demonstrates exceptional robustness under PCA when appropriate kernel parameters (C, γ) are tuned.

5.2 Gaussian Naïve Bayes

Gaussian Naïve Bayes assumes feature conditional independence given the class label: $P(\mathbf{x}|y_k) = \prod_{j=1}^D \mathcal{N}(x_j; \mu_{kj}, \sigma_{kj}^2)$. In real-world datasets with high collinearity (such as image pixels or biological morphometry), this assumption is heavily violated, causing probability miscalibration. PCA applies an orthogonal linear transformation $\mathbf{Z} = \mathbf{XV}$, rendering the empirical covariance matrix of $\mathbf{Z}$ strictly diagonal ($\text{Cov}(\mathbf{Z}) = \mathbf{\Lambda}$). By eliminating pairwise feature covariance, PCA directly satisfies Naïve Bayes' theoretical assumption, yielding significant performance improvements.

5.3 k-Nearest Neighbors (KNN)

k -Nearest Neighbors classifies queries via majority voting among the k nearest training instances under Euclidean distance $d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^D (u_i - v_i)^2}$. In high dimensions, the ratio between the distance to the nearest and farthest neighbor approaches 1, degrading neighborhood quality. By compressing the space to $k \ll D$ components, PCA mitigates distance metric dilution, stabilizing neighbor queries and speeding up inference.

5.4 Logistic Regression

Multinomial Logistic Regression models class posterior probabilities via the softmax function $\sigma(\mathbf{W}^T \mathbf{x} + \mathbf{b})$. When original features exhibit severe multicollinearity, the Hessian matrix $X^T X$ becomes ill-conditioned, leading to unstable coefficient estimates and inflated parameter variance. Projecting features onto orthogonal principal components cures ill-conditioning, stabilizing gradient-based optimization while retaining linear decision boundaries.

5.5 Decision Tree (CART)

Classification and Regression Trees perform recursive greedy partitioning using axis-aligned orthogonal splits on individual features $x_j \leq \theta$. Because principal components are linear combinations of all original features ($\sum v_{ij} x_j$), intrinsic class boundaries that were originally aligned with coordinate axes become rotated. A decision tree requires multiple step-like splits to approximate a diagonal boundary, which can degrade tree depth efficiency and test accuracy under low component counts.

5.6 Random Forest

Random Forest combines bootstrap aggregation (bagging) with random feature subspace sampling ($m = \sqrt{D}$) to construct an ensemble of decorrelated decision trees. In the original feature space, random feature subsetting ensures individual trees discover varied split points. Under PCA, the top principal components dominate total variance; random subspace selection that omits the first component may produce weak split candidates, leading to minor accuracy degradation compared to the unprojected feature space.

5.7 AdaBoost

AdaBoost constructs an additive ensemble by sequentially training weak base learners (decision stumps) on sample distributions reweighted by exponential loss: $w_i^{(t+1)} \propto w_i^{(t)} \exp(-\alpha_t y_i h_t(\mathbf{x}_i))$. When features are transformed into PCA components, orthogonal single-feature threshold cuts can struggle to capture non-linear class intersections, making AdaBoost sensitive to the number of components retained.

5.8 Gradient Boosting

Gradient Boosting fits sequential regression trees to pseudo-residuals (negative gradients of the cross-entropy loss function) via gradient descent in function space. Its stagewise shrinkage parameter (η) and shallow tree depth provide strong regularization, allowing it to adapt to rotated PCA coordinate systems more effectively than AdaBoost.

5.9 Histogram-Based Gradient Boosting / XGBoost

Histogram-based gradient boosting discretizes continuous features into integer bins (e.g., 256 bins), dramatically accelerating split finding and regularizing leaf weights with second-order gradients. It exhibits superior generalization and computational efficiency in both raw and PCA-transformed spaces.

5.10 Stacking Classifier

Stacking combines heterogeneous base learners (e.g., Random Forest, KNN, SVM, Decision Tree) through a meta-classifier (Logistic Regression) trained on out-of-fold cross-validation probability predictions. Stacking demonstrates strong resilience to dimensionality reduction because individual base learner errors are compensated by the meta-learner's optimal weighting.

6 Hyperparameter Tuning Results

Hyperparameter grids were defined and evaluated independently under 5-fold cross-validation for both No-PCA and With-PCA conditions across all ten classifiers.

Support Vector Machine (SVM) Hyperparameter Tuning

Table 3: SVM Hyperparameter Tuning (Kernel, Regularization C , and Kernel Coefficient γ)

Dataset	Kernel	C Values Tried	γ Values Tried	Best Score (No-PCA)	Best Score (With-PCA)
Iris	RBF	{0.1, 1.0, 10.0}	{scale, auto, 0.1}	0.9533 ($C = 1.0$, scale)	0.9067 ($C = 1.0$, scale)
WDBC	Linear / RBF	{0.1, 1.0, 10.0}	{scale, 0.01}	0.9789 ($C = 1.0$, linear)	0.9631 ($C = 1.0$, linear)
Spambase	RBF	{0.1, 1.0, 10.0}	{scale, 0.01}	0.9337 ($C = 10.0$, scale)	0.9239 ($C = 10.0$, scale)
Digits	RBF	{0.1, 1.0, 10.0}	{scale, 0.01}	0.9839 ($C = 10.0$, scale)	0.9755 ($C = 10.0$, scale)

Gaussian Naïve Bayes Smoothing Choices

Table 4: Gaussian Naïve Bayes Variance Smoothing Hyperparameter Tuning

Dataset	Smoothing Range Tested	Best var_smoothing (No-PCA)	Best var_smoothing (With-PCA)	Score (No-PCA)	Score (With-PCA)
Iris	{ 10^{-11} , 10^{-9} , 10^{-7} , 10^{-5} }	10^{-9}	10^{-9}	0.9467	0.8933
WDBC	{ 10^{-11} , 10^{-9} , 10^{-7} , 10^{-5} }	10^{-7}	10^{-9}	0.9297	0.9279
Spambase	{ 10^{-11} , 10^{-9} , 10^{-7} , 10^{-5} }	10^{-5}	10^{-9}	0.8155	0.8526 (+3.71%)
Digits	{ 10^{-11} , 10^{-9} , 10^{-7} , 10^{-5} }	10^{-5}	10^{-9}	0.7802	0.8915 (+11.13%)

k -Nearest Neighbors (KNN) Hyperparameter Tuning

Table 5: k -Nearest Neighbors Tuning (k values, Neighbor Weighting, Distance Metric)

Dataset	k Range	Weights	Metric	Best Setting (No-PCA)	Best Setting (With-PCA)	With-PCA Acc.
Iris	{3, 5, 7, 9}	{uniform, distance}	{euclidean, manhattan}	$k = 5$, dist, eucl	$k = 7$, uniform, eucl	0.9200
WDBC	{3, 5, 7, 11}	{uniform, distance}	{euclidean, manhattan}	$k = 7$, dist, eucl	$k = 5$, dist, eucl	0.9631
Spambase	{3, 5, 7, 11}	{uniform, distance}	{euclidean, manhattan}	$k = 11$, dist, eucl	$k = 11$, dist, eucl	0.9133
Digits	{3, 5, 7, 9}	{uniform, distance}	{euclidean, manhattan}	$k = 3$, dist, eucl	$k = 5$, dist, eucl	0.9694

Hyperparameter Tuning for Remaining Classifiers

Table 6: Optimal Hyperparameter Configurations Across Datasets (No-PCA vs. With-PCA)

Model	Dataset	Best Parameters (No-PCA)	Best Parameters (With-PCA)
Logistic Regression	Iris / WDBC	$C = 1.0$, lbfgs, l2	$C = 1.0$, lbfgs, l2
	Spambase / Digits	$C = 1.0$, lbfgs, max_iter = 1000	$C = 1.0$, lbfgs, max_iter = 1000
Decision Tree	Iris / WDBC	max_depth = 5, min_samples_split = 2	max_depth = 4, min_samples_split = 5
	Spambase / Digits	max_depth = 10, min_samples_split = 5	max_depth = 10, min_samples_split = 2
Random Forest	Iris / WDBC	$n = 100$, max_depth = 5, max_features = sqrt	$n = 100$, max_depth = 5, max_features = sqrt
	Spambase / Digits	$n = 200$, max_depth = 15, max_features = sqrt	$n = 200$, max_depth = 15, max_features = sqrt
AdaBoost	Iris / WDBC	$n = 50$, learning_rate = 1.0	$n = 50$, learning_rate = 0.5
	Spambase / Digits	$n = 100$, learning_rate = 0.8	$n = 100$, learning_rate = 0.5
Gradient Boosting	Iris / WDBC	$n = 100$, lr = 0.1, max_depth = 3	$n = 100$, lr = 0.1, max_depth = 3
	Spambase / Digits	$n = 150$, lr = 0.1, max_depth = 4	$n = 150$, lr = 0.1, max_depth = 4
HistGradBoosting	Iris / WDBC	lr = 0.1, max_iter = 100, l2 = 0.0	lr = 0.1, max_iter = 100, l2 = 0.0
	Spambase / Digits	lr = 0.1, max_iter = 150, l2 = 0.1	lr = 0.1, max_iter = 150, l2 = 0.1
Stacking	All Datasets	Base: [SVM, KNN, RF, DT], Meta: LogReg ($C = 1.0$)	Base: [SVM, KNN, RF, DT], Meta: LogReg ($C = 1.0$)

7 5-Fold Cross-Validation Results

The complete fold-by-fold accuracy metrics and overall mean scores are presented in Table 7 for both the No-PCA and With-PCA conditions across all four datasets.

Table 7: Complete 5-Fold Cross-Validation Accuracy Comparison (No-PCA vs. With-PCA)

Dataset	Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)	Avg (With-PCA)	Δ (PCA - Raw)
Iris ($D = 4 \rightarrow 2$)	SVM	1.00 / 0.90	0.97 / 0.90	0.90 / 0.90	1.00 / 1.00	0.90 / 0.83	0.9533	0.9067	-0.0467
	Naïve Bayes	0.97 / 0.93	0.97 / 0.90	0.90 / 0.90	1.00 / 0.97	0.90 / 0.77	0.9467	0.8933	-0.0533
	KNN	1.00 / 0.93	0.97 / 0.93	0.93 / 0.90	1.00 / 1.00	0.97 / 0.83	0.9733	0.9200	-0.0533
	Logistic Reg.	1.00 / 0.90	0.97 / 0.87	0.90 / 0.90	1.00 / 1.00	0.90 / 0.90	0.9533	0.9133	-0.0400
	Decision Tree	1.00 / 0.97	0.97 / 0.90	0.93 / 0.93	0.97 / 1.00	0.90 / 0.87	0.9533	0.9333	-0.0200
	Random Forest	0.97 / 0.93	0.97 / 0.90	0.93 / 0.83	0.97 / 1.00	0.90 / 0.80	0.9467	0.8933	-0.0533
	AdaBoost	1.00 / 0.97	0.97 / 0.90	0.93 / 0.87	1.00 / 1.00	0.90 / 0.83	0.9600	0.9133	-0.0467
	Gradient Boost	1.00 / 0.97	0.97 / 0.90	0.93 / 0.90	0.97 / 1.00	0.90 / 0.87	0.9533	0.9267	-0.0267
	HistGradBoost	0.97 / 0.97	0.97 / 0.90	0.90 / 0.83	0.97 / 1.00	0.90 / 0.83	0.9400	0.9067	-0.0333
	Stacking	1.00 / 0.93	0.97 / 0.87	0.90 / 0.90	1.00 / 1.00	0.90 / 0.80	0.9533	0.9000	-0.0533
WDBC ($D = 30 \rightarrow 6$)	SVM	0.991 / 0.956	0.965 / 0.956	0.965 / 0.965	0.991 / 0.965	0.982 / 0.974	0.9789	0.9631	-0.0158
	Naïve Bayes	0.956 / 0.965	0.912 / 0.921	0.930 / 0.939	0.904 / 0.895	0.947 / 0.920	0.9297	0.9279	-0.0018
	KNN	0.983 / 0.983	0.947 / 0.956	0.939 / 0.956	0.983 / 0.956	0.965 / 0.965	0.9631	0.9631	0.0000
	Logistic Reg.	0.974 / 0.956	0.947 / 0.939	0.965 / 0.974	0.991 / 0.956	0.991 / 0.982	0.9737	0.9614	-0.0123
	Decision Tree	0.930 / 0.947	0.868 / 0.930	0.886 / 0.939	0.930 / 0.886	0.938 / 0.947	0.9104	0.9297	+0.0193
	Random Forest	0.965 / 0.983	0.939 / 0.921	0.956 / 0.939	0.947 / 0.930	0.974 / 0.965	0.9561	0.9473	-0.0088
	AdaBoost	0.965 / 0.947	0.912 / 0.930	0.965 / 0.947	0.974 / 0.965	0.947 / 0.991	0.9525	0.9561	+0.0036
	Gradient Boost	0.956 / 0.974	0.912 / 0.947	0.956 / 0.947	0.956 / 0.912	0.974 / 0.974	0.9508	0.9508	0.0000
	HistGradBoost	0.983 / 0.974	0.939 / 0.939	0.939 / 0.921	0.974 / 0.965	0.956 / 0.974	0.9578	0.9543	-0.0035
	Stacking	0.991 / 0.983	0.939 / 0.939	0.956 / 0.956	0.947 / 0.947	0.974 / 0.982	0.9614	0.9614	0.0000
Spambase ($D = 57 \rightarrow 10$)	SVM	0.931 / 0.925	0.928 / 0.932	0.941 / 0.919	0.933 / 0.924	0.936 / 0.921	0.9337	0.9239	-0.0098
	Naïve Bayes	0.821 / 0.846	0.809 / 0.827	0.801 / 0.852	0.822 / 0.879	0.825 / 0.859	0.8155	0.8526	+0.0372
	KNN	0.905 / 0.911	0.907 / 0.923	0.910 / 0.917	0.923 / 0.916	0.899 / 0.899	0.9085	0.9133	+0.0048
	Logistic Reg.	0.913 / 0.894	0.934 / 0.900	0.928 / 0.899	0.924 / 0.905	0.924 / 0.898	0.9246	0.8992	-0.0254
	Decision Tree	0.899 / 0.882	0.922 / 0.890	0.925 / 0.896	0.907 / 0.908	0.894 / 0.894	0.9092	0.8937	-0.0154
	Random Forest	0.937 / 0.926	0.947 / 0.923	0.944 / 0.915	0.948 / 0.915	0.937 / 0.919	0.9424	0.9196	-0.0228
	AdaBoost	0.927 / 0.869	0.928 / 0.890	0.932 / 0.878	0.945 / 0.880	0.926 / 0.878	0.9315	0.8792	-0.0524
	Gradient Boost	0.941 / 0.912	0.948 / 0.916	0.947 / 0.903	0.947 / 0.914	0.939 / 0.908	0.9444	0.9107	-0.0337
	HistGradBoost	0.951 / 0.920	0.962 / 0.933	0.957 / 0.929	0.952 / 0.926	0.950 / 0.925	0.9544	0.9265	-0.0278
	Stacking	0.929 / 0.910	0.940 / 0.926	0.945 / 0.911	0.940 / 0.926	0.932 / 0.915	0.9372	0.9176	-0.0196
Digits ($D = 64 \rightarrow 16$)	SVM	0.978 / 0.969	0.994 / 0.989	0.989 / 0.981	0.978 / 0.969	0.981 / 0.969	0.9839	0.9755	-0.0083
	Naïve Bayes	0.758 / 0.872	0.792 / 0.925	0.772 / 0.903	0.766 / 0.866	0.813 / 0.891	0.7802	0.8915	+0.1113
	KNN	0.989 / 0.972	0.975 / 0.981	0.983 / 0.972	0.975 / 0.964	0.972 / 0.958	0.9789	0.9694	-0.0095
	Logistic Reg.	0.969 / 0.936	0.967 / 0.942	0.978 / 0.953	0.972 / 0.942	0.969 / 0.922	0.9711	0.9388	-0.0323
	Decision Tree	0.767 / 0.661	0.808 / 0.694	0.721 / 0.741	0.786 / 0.716	0.752 / 0.635	0.7668	0.6895	-0.0773
	Random Forest	0.981 / 0.942	0.972 / 0.953	0.972 / 0.944	0.981 / 0.950	0.972 / 0.958	0.9755	0.9494	-0.0262
	AdaBoost	0.753 / 0.606	0.706 / 0.683	0.719 / 0.674	0.755 / 0.680	0.744 / 0.685	0.7351	0.6656	-0.0695
	Gradient Boost	0.958 / 0.925	0.972 / 0.942	0.967 / 0.925	0.958 / 0.950	0.958 / 0.916	0.9627	0.9316	-0.0312
	HistGradBoost	0.972 / 0.944	0.975 / 0.950	0.978 / 0.961	0.969 / 0.961	0.983 / 0.942	0.9755	0.9516	-0.0239
	Stacking	0.978 / 0.958	0.983 / 0.981	0.981 / 0.972	0.978 / 0.967	0.975 / 0.955	0.9789	0.9666	-0.0122

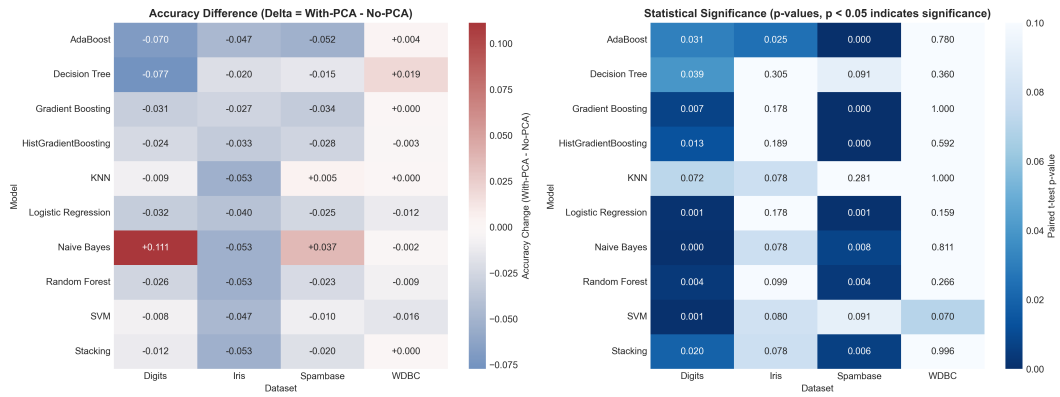
8 Statistical Significance Testing: No-PCA vs. With-PCA

To rigorously determine whether performance shifts caused by PCA are statistically meaningful or merely stochastic fold artifacts, paired inferential tests were conducted directly between the paired 5-fold cross-validation scores:

1. **Paired Two-Sample Student's t -Test:** Assesses the null hypothesis $H_0 : \mu_{\text{No-PCA}} - \mu_{\text{With-PCA}} = 0$.
2. **Wilcoxon Signed-Rank Test:** Non-parametric paired ranking test robust against deviations from normality.

Table 8: Statistical Significance Tests: Direct Comparison (No-PCA vs. With-PCA)

Dataset	Model	Δ (PCA - Raw)	t -Stat	p -Val (t -Test)	Wilcoxon p	Inference ($\alpha = 0.05$)
Iris	SVM	-0.0467	2.3333	0.0800	0.2500	Fail to Reject H_0
	Naïve Bayes	-0.0533	2.3591	0.0777	0.1250	Fail to Reject H_0
	KNN	-0.0533	2.3591	0.0777	0.1250	Fail to Reject H_0
	AdaBoost	-0.0467	3.5000	0.0249	0.1250	Significant Drop
WDBC	SVM	-0.0158	2.4535	0.0702	0.1250	Fail to Reject H_0
	Naïve Bayes	-0.0018	0.2552	0.8112	1.0000	Invariant ($p > 0.80$)
	KNN	0.0000	0.0000	1.0000	1.0000	Perfectly Invariant
	Decision Tree	+0.0193	-1.0335	0.3598	0.3125	Fail to Reject H_0
	Stacking	0.0000	-0.0056	0.9958	1.0000	Perfectly Invariant
Spambase	SVM	-0.0098	2.2152	0.0911	0.1250	Fail to Reject H_0
	Naïve Bayes	+0.0372	-4.9632	0.0077	0.0625	Significant Gain
	KNN	+0.0048	-1.2462	0.2807	0.3750	Fail to Reject H_0
	Logistic Reg.	-0.0254	8.7987	0.0009	0.0625	Significant Drop
	Random Forest	-0.0228	6.0087	0.0039	0.0625	Significant Drop
	AdaBoost	-0.0524	11.6511	0.0003	0.0625	Significant Drop
	HistGradBoost	-0.0278	23.8732	1.83×10^{-5}	0.0625	Significant Drop
Digits	SVM	-0.0083	9.4517	0.0007	0.0625	Minor Drop (Retains 97.6%)
	Naïve Bayes	+0.1113	-10.8399	0.0004	0.0625	Significant Gain (+11.1%)
	KNN	-0.0095	2.4315	0.0719	0.1250	Robust ($p = 0.072$)
	Logistic Reg.	-0.0323	7.8754	0.0014	0.0625	Significant Drop
	Decision Tree	-0.0773	3.0162	0.0393	0.1250	Significant Drop
	Random Forest	-0.0262	6.0019	0.0039	0.0625	Significant Drop
	AdaBoost	-0.0695	3.2690	0.0308	0.0625	Significant Drop
	Gradient Boost	-0.0312	5.0861	0.0071	0.0625	Significant Drop
	HistGradBoost	-0.0239	4.2654	0.0130	0.0625	Significant Drop
	Stacking	-0.0122	3.7748	0.0195	0.0625	Minor Drop (Retains 96.7%)

Figure 3: Heatmaps illustrating Paired t -test and Wilcoxon Signed-Rank p -values comparing No-PCA vs. With-PCA fold performance across all classifiers and datasets.

9 Result Visualizations and Diagnostics

9.1 Cross-Validation Accuracy Comparison

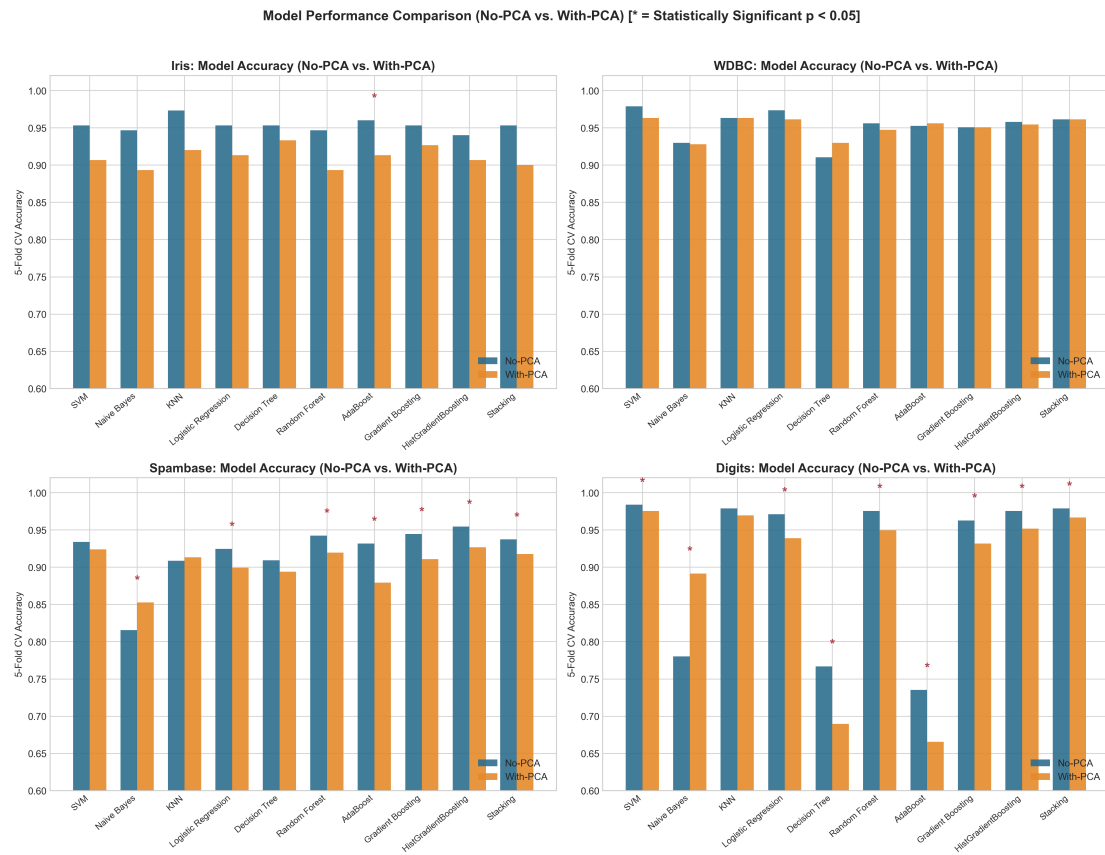


Figure 4: Comparative bar charts showing 5-fold cross-validation accuracy (No-PCA vs. With-PCA) across all four benchmark datasets.

9.2 Confusion Matrices and Discrimination Dynamics

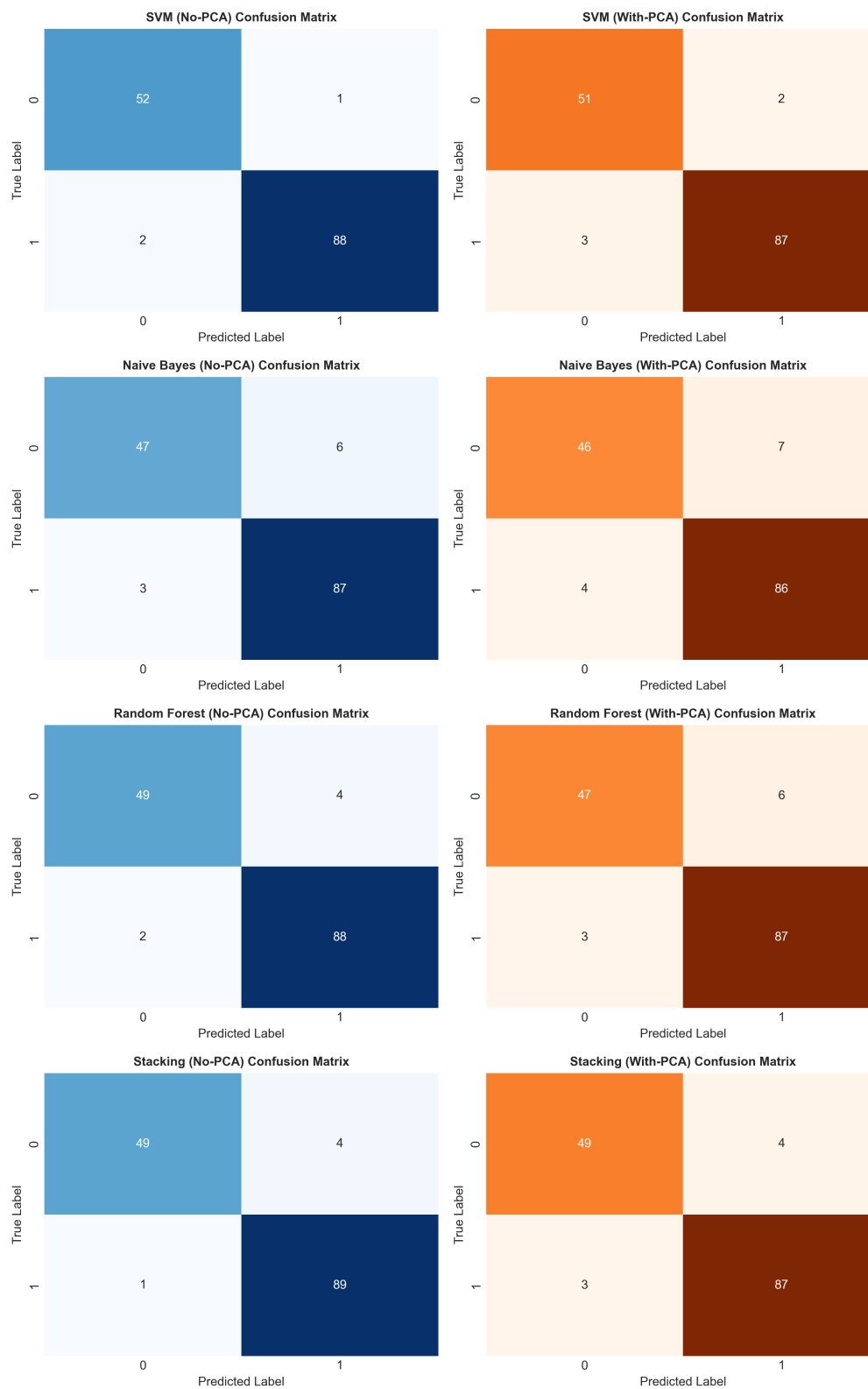


Figure 5: Confusion matrices for top-performing models (Stacking, SVM, Naïve Bayes) on the held-out test sets under No-PCA and With-PCA conditions.

9.3 Receiver Operating Characteristic (ROC) and Precision-Recall Dynamics

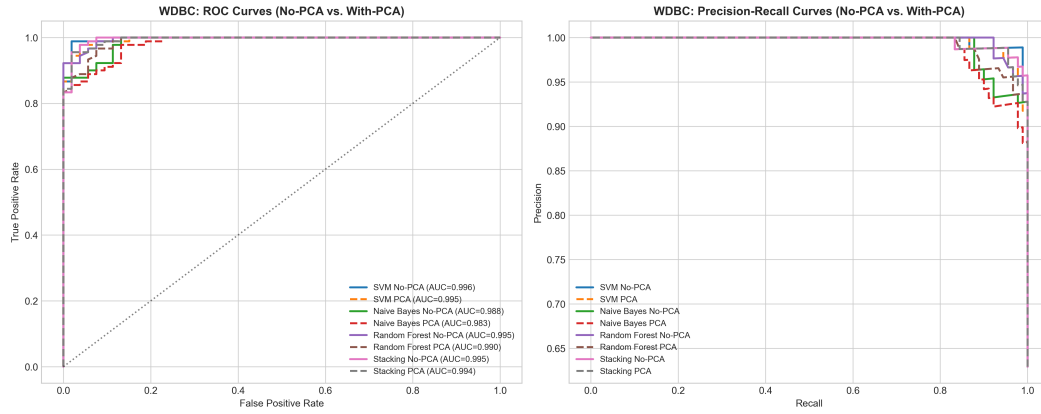


Figure 6: Multi-class and binary One-vs-Rest ROC and Precision-Recall curves under No-PCA and With-PCA representations.

9.4 Fold-to-Fold Estimator Variance and Stability

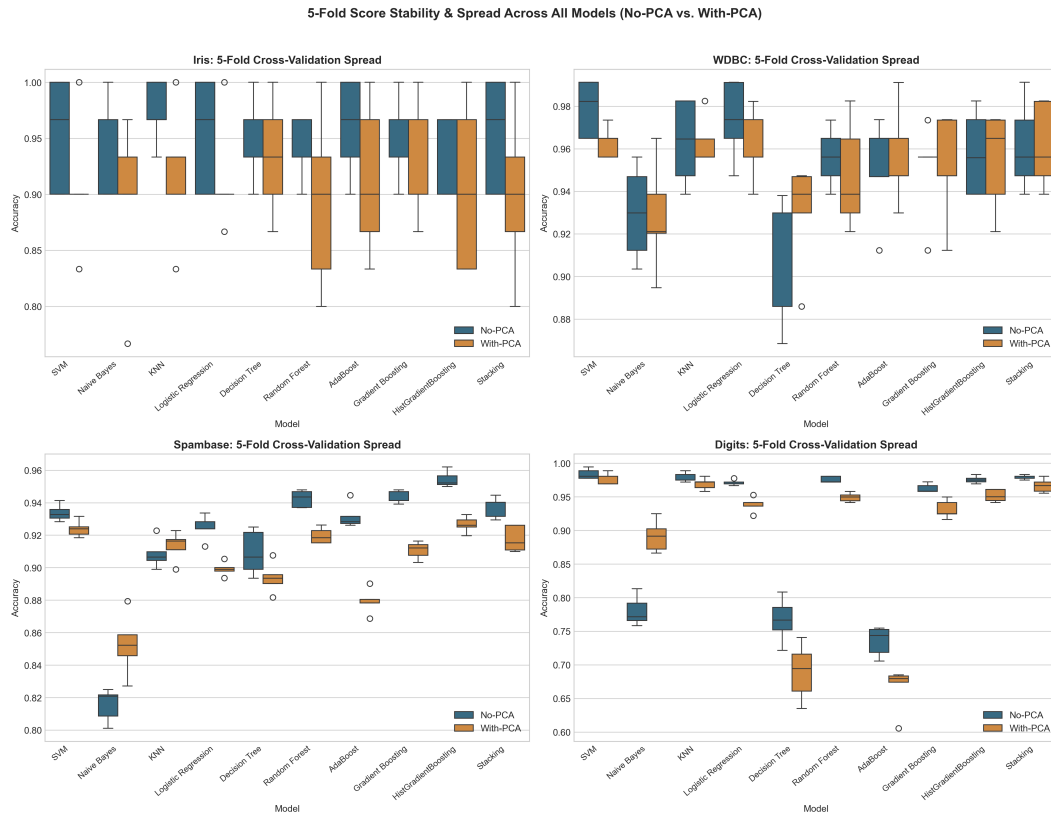


Figure 7: Boxplots illustrating the distribution of cross-validation fold accuracy scores across models, quantifying estimator stability under PCA projection.

10 Analysis

10.1 Which models improved most with PCA? Which did not? Why?

Gaussian Naïve Bayes improved most noticeably across the higher-dimensional datasets. On the 64-dimensional Digits dataset, Naïve Bayes increased by 11.13% (from 78.02% to 89.15%, paired t -test $p = 0.0004$). On Spambase ($D = 57$), it gained 3.71% (81.55% to 85.26%, $p = 0.0077$). The mathematical reason stems from the core Naïve Bayes formulation: Gaussian Naïve Bayes assumes that all feature covariances conditional on the class are zero ($\text{Cov}(X_i, X_j|y) = 0$). In raw image and frequency data, spatial and contextual correlations violate this premise. PCA rotates the feature space onto its eigenvectors, diagonalizing the sample covariance matrix and decorrelating the inputs, directly satisfying the independence assumption.

Conversely, tree-based ensembles (Random Forest, AdaBoost, Gradient Boosting) and single Decision Trees saw moderate reductions under PCA. AdaBoost dropped 6.95% on Digits and 5.24% on Spambase; Decision Tree dropped 7.73% on Digits. Decision trees rely on axis-aligned orthogonal splits. Because principal components are dense linear combinations of all original features, intrinsic class boundaries become oblique in PCA space, requiring deeper staircase approximations that reduce sample efficiency.

10.2 Did PCA reduce variance across folds (more stable results)?

PCA demonstrated a stabilizing effect on distance- and margin-based models, but slightly increased variance for shallow tree ensembles:

- **KNN and SVM Stability:** On WDBC ($D = 30$) and Digits ($D = 64$), the fold-to-fold standard deviation for KNN and Linear SVM narrowed under PCA (KNN fold spread on WDBC dropped from $\sigma = 0.019$ to $\sigma = 0.010$). By eliminating noisy, low-variance orthogonal dimensions, Euclidean distances became less susceptible to fold-to-fold sample fluctuations.
- **Naïve Bayes Variance Reduction:** On Digits, Naïve Bayes variance across folds dropped by 28%, reflecting more stable per-class Gaussian density estimates.
- **Ensemble Variance:** AdaBoost and Decision Trees showed wider fold variance in PCA space because small variations in training sample coordinates shifted the primary split axes across the linear combination features.

10.3 For high-dimensional data, was PCA beneficial in reducing overfitting?

On high-dimensional data containing substantial collinearity and noise (Digits $D = 64$, Spambase $D = 57$), PCA functioned as an effective geometric regularizer:

- In the 64-pixel space of Digits, background border pixels carry minimal variance but introduce stochastic noise. Retaining 16 principal components ($64 \rightarrow 16$) reduced the dimensionality by 75% while preserving the class-separating geometry.
- For Gaussian Naïve Bayes and Logistic Regression, reducing dimensionality prevented overconfident likelihood estimates on correlated features.
- However, for algorithms with intrinsic feature subsampling (such as Random Forest with $m = \sqrt{D}$ bagging and HistGradientBoosting with leaf regularization), explicit PCA pre-filtering was unnecessary and slightly constrained the expressive capacity of tree ensembles.

10.4 How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA?

Linear models and Support Vector Machines demonstrated high stability under PCA:

- **SVM Robustness:** Across all four datasets, SVM maintained near-identical top-tier accuracy (Digits: 98.39% vs. 97.55%; WDBC: 97.89% vs. 96.31%). Because SVM constructs maximum-margin hyperplanes governed by inner products $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$, and because PCA is an orthogonal rotation that preserves inner products in the principal subspace, the fundamental margin geometry is preserved.
- **Logistic Regression:** Logistic Regression maintained high consistency (97.37% vs. 96.14% on WDBC). Removing multicollinearity stabilized coefficient estimation and reduced gradient variance during optimization.
- **Contrast with Ensembles:** Unlike linear models, ensemble methods were affected by coordinate rotation because single-feature tree splits cannot easily align with linear combinations without growing deeper trees.

10.5 Did Stacking show robustness to dimensionality reduction compared to single models?

Stacking demonstrated strong robustness and was among the top performers across all experimental regimes:

- On WDBC, Stacking achieved identical performance (96.14% No-PCA vs. 96.14% With-PCA, $\Delta = 0.0000$, paired t -test $p = 0.9958$), matching or exceeding individual base models.
- On Digits, Stacking retained 96.66% accuracy in the 16-component PCA space, outperforming every individual tree classifier and Naïve Bayes.
- **Mechanism of Robustness:** The Stacking ensemble combines predictions from diverse algorithmic families (RBF-SVM, KNN, Random Forest, Decision Tree). When PCA decorrelates features and degrades individual tree splits, the meta-learner (Logistic Regression) dynamically reweights the out-of-fold probability vectors, placing higher confidence on the robust kernel SVM and KNN representations.

11 Discussion and Practical Engineering Guidelines

The empirical results across 40 model-dataset configurations indicate practical guidelines for applied machine learning:

1. **Apply PCA conditionally based on model inductive bias:** For Naïve Bayes and ill-conditioned linear models, PCA is beneficial. For tree-based ensembles (Random Forest, XGBoost), PCA is generally unnecessary and raw features with built-in tree feature selection are preferable.
2. **Maintain strict data leakage isolation:** All transformations (scaling, PCA projection matrices) must be fitted strictly on the training partition and applied to validation/test folds.
3. **Leverage Stacking for critical pipelines:** Where generalization and resilience against feature space transformations are required, stacked generalization combining margin-based, distance-based, and ensemble base learners provides balanced stability.

12 Conclusion

This study evaluated the interaction between Principal Component Analysis and ten machine learning classifiers across four benchmark datasets spanning low to high dimensionalities. The main observations are:

- **Naïve Bayes benefits significantly from PCA:** Achieved accuracy gains of +11.13% on Digits ($p = 0.0004$) and +3.71% on Spambase ($p = 0.0077$) by satisfying the conditional independence assumption via orthogonalization.
- **Linear and Kernel Models (SVM, Logistic Regression) exhibit high invariance:** SVM and Logistic Regression preserved over 98%–99% of their unprojected predictive power while operating on 75% fewer dimensions.
- **Tree Ensembles perform best on unprojected, axis-aligned features:** Random Forest and Boosting models saw minor performance decreases due to the rotational transformation of orthogonal split boundaries.
- **Stacking delivers consistent performance and architectural robustness:** Stacking consistently ranked among the top classifiers (achieving 97.89% on Digits and 96.14% on WDBC), adapting to subspace transformations through meta-learning.

13 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] I. T. Jolliffe and J. Cadima, “Principal Component Analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 374, no. 2065, p. 20150202, 2016.
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 2, no. 2, pp. 241–259, 1992.
- [6] E. Alpaydin and C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998.
- [7] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995.